

企业级项目3- 物联网设备管理系统

1.1 项目展示

1.2 项目要求

1.3 程序设计

1. 绘制背景图片

```
// 绘制背景图片
QPalette PAllbackground = this->palette();
QImage ImgAllbackground(QString::fromUtf8(":/images/back.png"));
QImage pix = ImgAllbackground.scaled(this->size(),Qt::IgnoreAspectRatio);
PAllbackground.setBrush(QPalette::Window, QBrush(pix));
this->setPalette(PAllbackground);
```

2. listwidget的样式表

```
// 设置 listwidget 的样式表
ui->listWidget->setStyleSheet("QListWidget { background: transparent;"
                                "border: none; "
                                "outline: none;"
                                "font-size: 18px; "
                                "color: white; "
                                "font-weight: bold;"
                                "padding-top: 30px;}"

                                "QListWidget::item { min-height: 50px;"
                                "border: none; }"

                                "QListWidget::item:selected {min-height: 50px;"
                                "background: #5354cb;"
                                "border: none; "
                                "border-radius: 10px; }"
                                );
```

3. tabWidget 的样式表

```
// 设置 tabwidget 透明
ui->tabWidget->setStyleSheet("QTabWidget::pane { background: transparent;border: none; }");
ui->tabWidget->tabBar()->hide() ; // 隐藏标签项
ui->tabWidget->setCurrentIndex(0);
```

4. 获取串口代码

```
QList<QSerialPortInfo> serialPorts = QSerialPortInfo::availablePorts() ;
foreach (QSerialPortInfo port, serialPorts) {
    qDebug() << "串口名: " << port.portName() ; // 串口名称
    qDebug() << "串口描述: " << port.description() ; // 串口描述
    ui->comboBox->addItem(port.portName()+" #"+port.description(),QVariant(port.portName()));
}
```

5. 串口发送与接收实现

```
//数组首地址、数组长度-2（不包含校验码）
unsigned int make_crc(unsigned char *puchMsg,unsigned int usDataLen)
{
    unsigned char i;
    unsigned int crc=0xFFFF;
    while(usDataLen--!=0)
    {
        crc=crc^(*puchMsg);
        for(i=0;i<8;i++){if((crc&0x0001)==1){crc=crc>>1;crc^=0xA001;}else{crc=crc>>1;}}
        puchMsg++;
    }
    return(crc);
}

// 成功读取返回： true
// 失败读取返回： false
bool widget::cmdReadBytes(quint8 readData[],quint8 n)
{
    // 在 115200 的波特率下发送 100 字节需要约 6.944 毫秒。
    if (!serial->waitForReadyRead(3000)) return false;
    QByteArray data = serial->readAll();
    while (serial->waitForReadyRead(10))
        data += serial->readAll();

    for(qint32 i=0;i<data.size();i++)
    {
        readData[i] = quint8(data.at(i)) ;// 数据入队
        //qDebug("%#x",quint8(data.at(i))) ;
    }
    quint16 recv_crc = (readData[n-1] << 8) | readData[n-2] ;
    quint16 calc_crc = make_crc(readData,n-2);// 计算出的crc
    // qDebug("recv_crc = %#x",recv_crc) ;
    // qDebug("calc_crc = %#x",calc_crc) ;
    if( recv_crc == calc_crc )
    {
        return true;
    }
    else
    {
        static qint32 count =0 ;
        qDebug("crc校验失败次数:%d",++count) ;
        return false;
    }
}

// 返回真： 表示命令发送成功
// 返回假： 表示命令发送失败
bool widget::cmdStatus(quint8 src_cmd[],quint8 n)
{
    quint8 dst_cmd[128];

    // 在 115200 的波特率下发送 100 字节需要约 6.944 毫秒。
    if (!serial->waitForReadyRead(1000)) return false;
    QByteArray data = serial->readAll();
    while (serial->waitForReadyRead(10))
        data += serial->readAll();

    for(qint32 i=0;i<data.size();i++)
    {
        dst_cmd[i] = quint8(data.at(i)) ;// 数据入队
        qDebug() << "->:" << quint8(data.at(i));
    }
    if( memcmp(src_cmd,dst_cmd,n) == 0 )
    {
        return true;
    }
    else
    {
        qDebug() << "cmdStatus memcpy failed";
        return false;
    }
}
```

6. 获取CAN设备代码

```
candle_list_handle clist;
uint8_t num_interfaces;
candle_handle dev;
if (candle_list_scan(&clist)) {
    if (candle_list_length(clist, &num_interfaces)) {
        for (uint8_t i=0; i<num_interfaces; i++) {
            if (candle_dev_get(clist, i, &dev)) {
                CanDevice device;
                device.id = i;
                device.handle = dev;
                device.name = "candle" + QString::number(device.id & 0xFF);
                device.desc = QString::fromStdString(candle_dev_get_path(dev));
                ui->comboBox_can->addItem(device.name);
                devices.append(device);
            }
        }
    }
    candle_list_free(clist);
}
```

```
// 设置信号处理函数
connect(this,&V9CANopen::SigReceiveCanData,this,&V9CANopen::SlotReceiveCanData);
```

7. 定义CanDevice 设备

```
struct CanDevice
{
    candle_handle handle;
    uint32_t id;
    QString name;
    QString desc; // 驱动描述
};
```

8. 打开can设备

```
void V9CANopen::on_pushButton_open_clicked(bool checked)
{
    qDebug() << "checked=" << checked;
    if(checked) // 为真 打开设备
    {
        CanDevice device = devices[ui->comboBox_can->currentIndex()];
        if (!candle_dev_open(device.handle)) {
            // TODO what?
            qDebug()<<"error 1";
            return;
        }

        // 读取can的属性信息
        candle_capability_t caps;
        if (!candle_channel_get_capabilities(device.handle, 0, &caps)) {
            qDebug()<<"error 2";
            return;
        }
        else
        {
            qDebug()<<"scan attribute:" << caps.fc1k_can << caps.brp_min << caps.brp_max << caps.brp_inc << caps.feature;
        }
        // 500000, 875 , 设置波特率 500K
        candle_bittiming_t timing;
        timing.brp = 6;
        timing.phase_seg1 = 12;
        timing.phase_seg2 = 2;
        timing.sjw = 1;
        timing.prop_seg = 1;
        if(!candle_channel_set_timing(device.handle, 0, &timing))
        {
            qDebug()<<"error 3";
            return;
        }

        uint32_t t_dev;
        candle_dev_get_timestamp_us(device.handle, &t_dev);

        uint32_t flags = 0;
        // flags |= CANDLE_MODE_LISTEN_ONLY;
        // flags |= CANDLE_MODE_ONE_SHOT;
        // flags |= CANDLE_MODE_TRIPLE_SAMPLE;
        if(candle_channel_start(device.handle, 0, flags) )
        {
            qDebug() <<device.name << "open scuessed" ;
        }
        else
        {
            qDebug() <<device.name << "open failed" ;
        }

        // 设置信号处理函数 SigReceiveCanData

        // 开启1个线程 , 保存函数
        future = QtConcurrent::run(this,&V9CANopen::readMessage);

        // 启动定时器
        if(timer != nullptr)
        {
            timer->start(2000);
        }

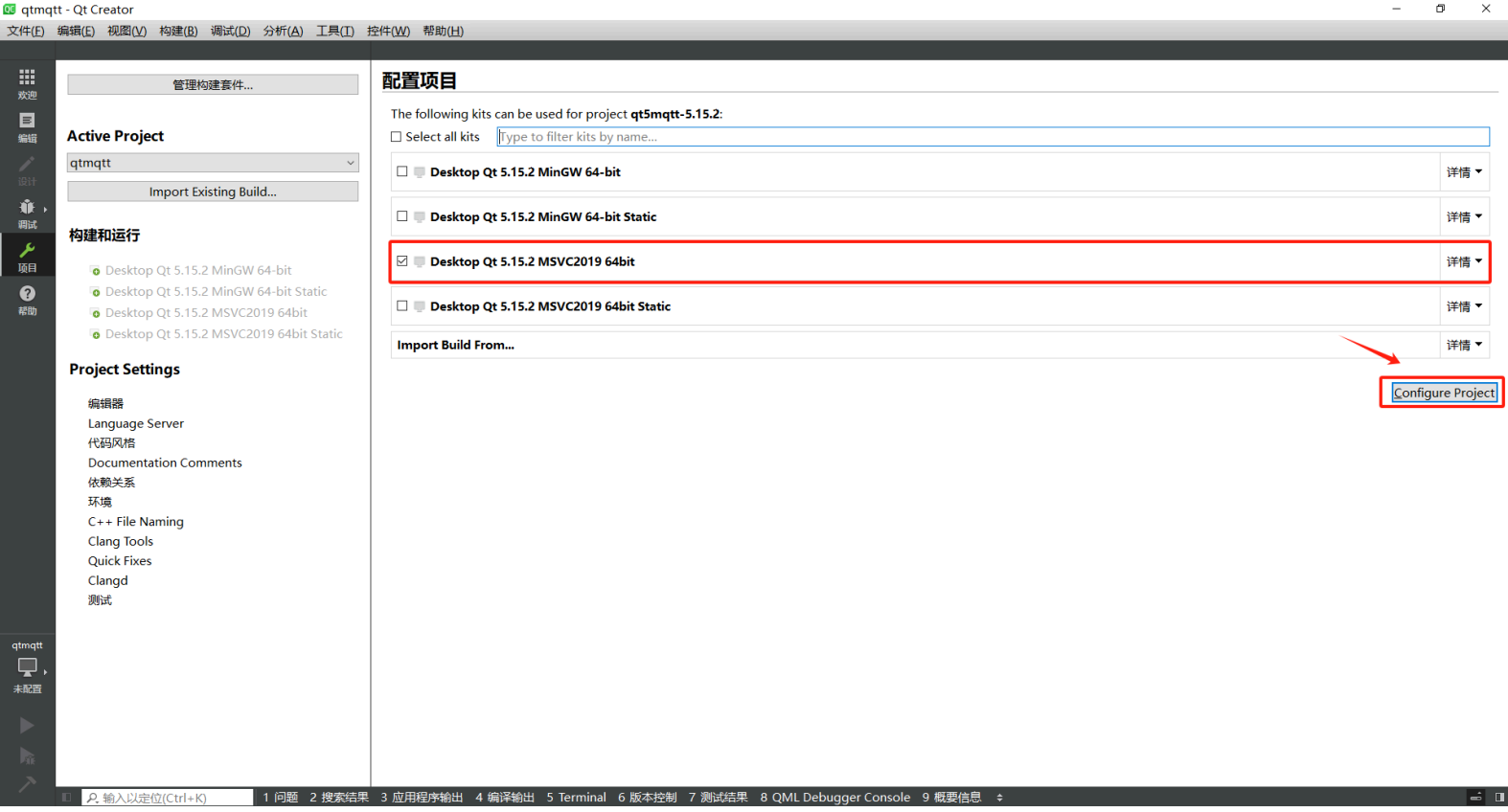
    }
    else // 为假 关闭设备
    {
        CanDevice device = devices[ui->comboBox_can->currentIndex()];
        future.cancel(); // 取消线程, 关闭线程
        future.waitForFinished();
        if (future.isCanceled())
        {
            qDebug() << "The task has been successfully canceled.";
        }
        else
        {
            qDebug() << "The task was not canceled or has not finished yet.";
        }
        qDebug()<<"stop :" << candle_channel_stop(device.handle, 0);
        qDebug()<<"close :" <<candle_dev_close(device.handle);
    }
}
```

1.5 编译MQTT库

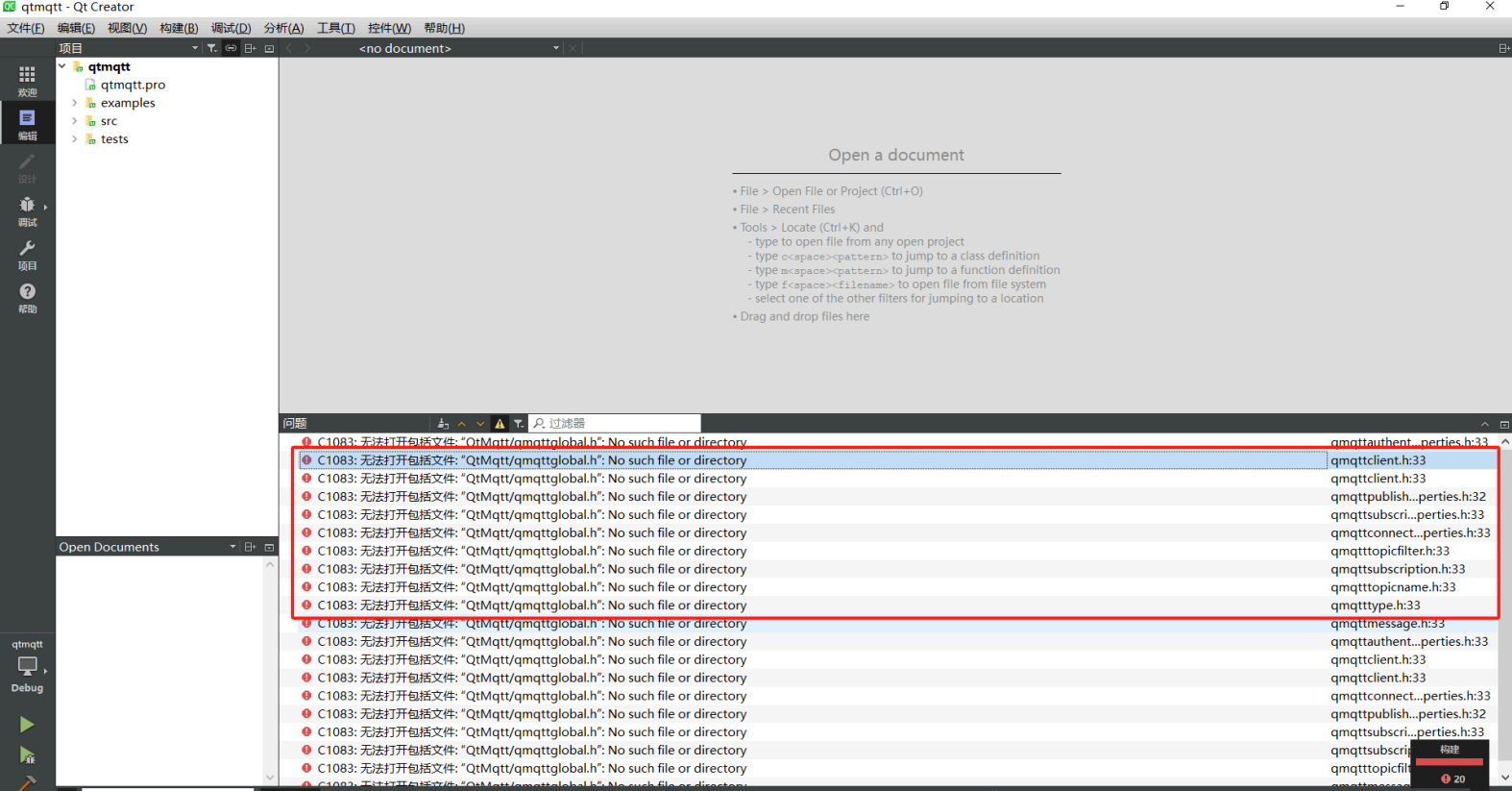
1. window下编译Mqtt库

<https://github.com/qt/qtt/tree/5.15.2>

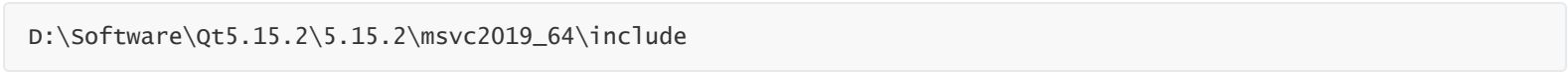
打开QT工程，使用MSVC2019开发套件。



编译QT工程，会出现错误，如下：



解决办法，QT安装路径的对应include路径下新建一个文件夹QtMqtt，目录如下：



根据自己的目录进行创建

名称	修改日期	类型	大小
QtDesigner	2020/11/6 17:04	文件夹	
QtDesignerComponents	2020/11/6 17:04	文件夹	
QtDeviceDiscoverySupport	2020/11/6 13:29	文件夹	
QtEdidSupport	2020/11/6 13:29	文件夹	
QtEglSupport	2020/11/6 13:29	文件夹	
QtEventDispatcherSupport	2020/11/6 13:29	文件夹	
QtFbSupport	2020/11/6 13:29	文件夹	
QtFontDatabaseSupport	2020/11/6 13:29	文件夹	
QtGamepad	2020/11/6 17:03	文件夹	
QtGui	2020/11/6 13:28	文件夹	
QtHelp	2020/11/6 17:05	文件夹	
QtLocation	2020/11/10 18:50	文件夹	
QtMqtt	2024/1/9 20:17	文件夹	
QtMultimedia	2020/11/6 17:08	文件夹	
QtMultimediaQuick	2020/11/6 17:08	文件夹	
QtMultimediaWidgets	2020/11/6 17:08	文件夹	
QtNetwork	2020/11/6 13:27	文件夹	
QtNfc	2020/11/6 17:02	文件夹	
QtOpenGL	2020/11/6 13:29	文件夹	
QtOpenGLExtensions	2020/11/6 13:29	文件夹	

复制 qtmqtt\qt5mqtt-5.15.2\src\mqtt 下的 *.h 文件到 QtMqtt 目录下。

YShengli > 02-ClassNotes > emb2307 > 13-linuxdriver > 28-qtmqtt > qt5mqtt-5.15.2 > src > mqtt > 在 mqtt 中搜索

名称	修改日期	类型	大小
doc	2024/1/9 19:20	文件夹	
qmqttauthenticationproperties.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtclient.h	2020/10/27 21:07	C++ Header file	9 KB
qmqtclient_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtconnection_p.h	2020/10/27 21:07	C++ Header file	7 KB
qmqtconnectionproperties.h	2020/10/27 21:07	C++ Header file	6 KB
qmqtconnectionproperties_p.h	2020/10/27 21:07	C++ Header file	4 KB
qmqtcontrolpacket_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtglobal.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtmessage.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtmessage_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtublishproperties.h	2020/10/27 21:07	C++ Header file	4 KB
qmqtublishproperties_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtsubscription.h	2020/10/27 21:07	C++ Header file	4 KB
qmqtsubscription_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtsubscriptionproperties.h	2020/10/27 21:07	C++ Header file	3 KB
qmqttopicfilter.h	2020/10/27 21:07	C++ Header file	4 KB
qmqttopicname.h	2020/10/27 21:07	C++ Header file	4 KB
qmqttype.h	2020/10/27 21:07	C++ Header file	3 KB
qmqttauthenticationproperties.cpp	2020/10/27 21:07	C++ Source file	4 KB
qmqtclient.cpp	2020/10/27 21:07	C++ Source file	31 KB
qmqtconnection.cpp	2020/10/27 21:07	C++ Source file	77 KB
qmqtconnectionproperties.cpp	2020/10/27 21:07	C++ Source file	19 KB
qmqtcontrolpacket.cpp	2020/10/27 21:07	C++ Source file	4 KB
qmqtmessage.cpp	2020/10/27 21:07	C++ Source file	6 KB
qmqtublishproperties.cpp	2020/10/27 21:07	C++ Source file	10 KB
qmqtsubscription.cpp	2020/10/27 21:07	C++ Source file	7 KB
qmqtsubscriptionproperties.cpp	2020/10/27 21:07	C++ Source file	5 KB
qmqttopicfilter.cpp	2020/10/27 21:07	C++ Source file	10 KB
qmqttopicname.cpp	2020/10/27 21:07	C++ Source file	6 KB
qmqttype.cpp	2020/10/27 21:07	C++ Source file	14 KB
mqtt.pro	2020/10/27 21:07	Qt Project file	2 KB

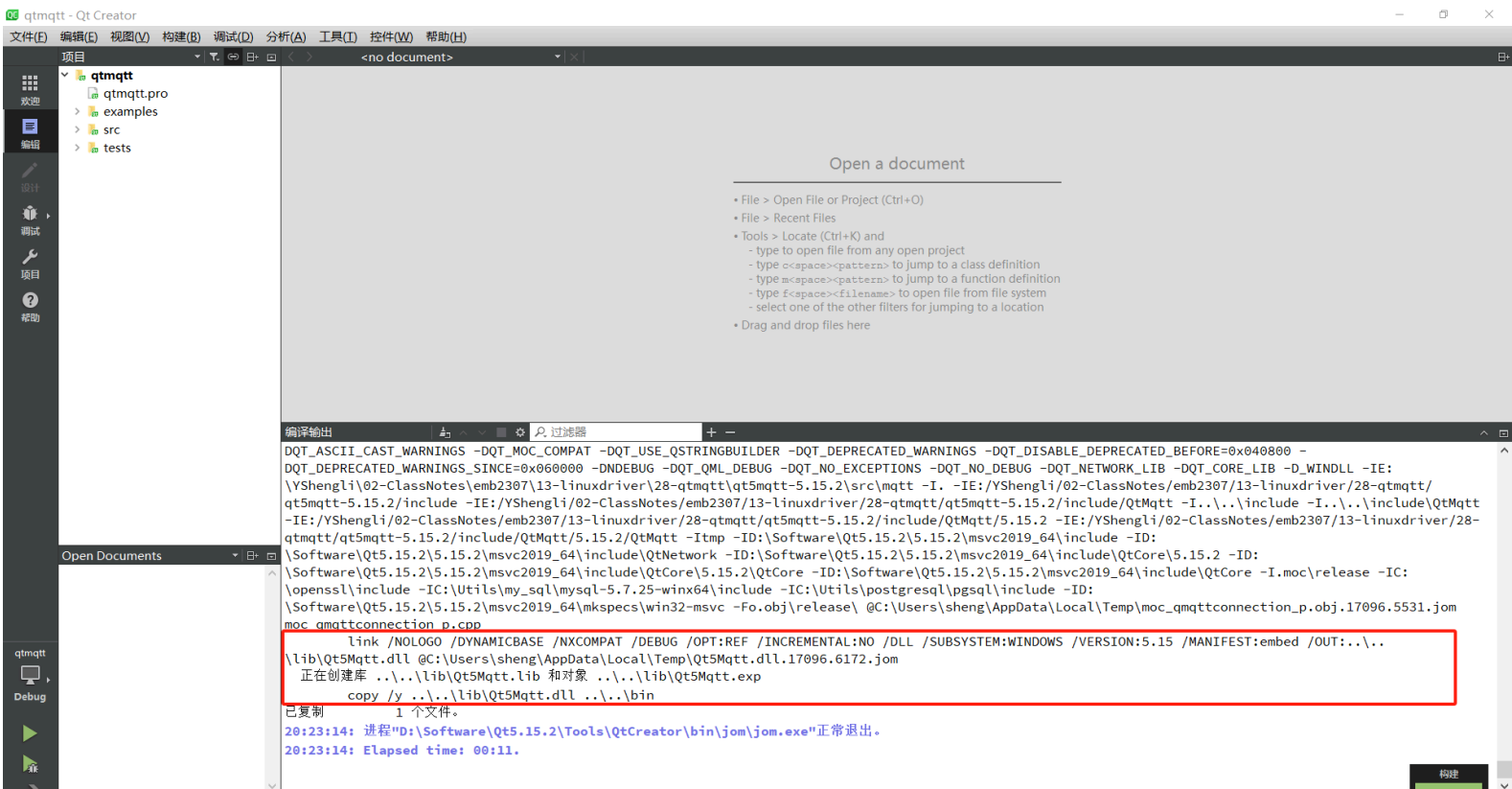
QtMqtt目录如下：

Software > Qt5.15.2 > 5.15.2 > msvc2019_64 > include > QtMqtt

在 QtMqtt 中搜索

名称	修改日期	类型	大小
qmqttauthenticationproperties.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtclient.h	2020/10/27 21:07	C++ Header file	9 KB
qmqtclient_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtconnection_p.h	2020/10/27 21:07	C++ Header file	7 KB
qmqtconnectionproperties.h	2020/10/27 21:07	C++ Header file	6 KB
qmqtconnectionproperties_p.h	2020/10/27 21:07	C++ Header file	4 KB
qmqtcontrolpacket_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtglobal.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtmessage.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtmessage_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtublishproperties.h	2020/10/27 21:07	C++ Header file	4 KB
qmqtublishproperties_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtsubscription.h	2020/10/27 21:07	C++ Header file	4 KB
qmqtsubscription_p.h	2020/10/27 21:07	C++ Header file	3 KB
qmqtsubscriptionproperties.h	2020/10/27 21:07	C++ Header file	3 KB
qmqttopicfilter.h	2020/10/27 21:07	C++ Header file	4 KB
qmqttopicname.h	2020/10/27 21:07	C++ Header file	4 KB
qmqttype.h	2020/10/27 21:07	C++ Header file	3 KB

这时候再次编译（Release模式）QtMqtt源码，编译完成后得到以下文件。



bin目录下就是我们要的库文件：

li > 02-ClassNotes > emb2307 > 13-linuxdriver > 28-qtmqtt > build-qtmqtt-Desktop_Qt_5_15_2_MSVC2019_64bit-Release > bin

在 bin 中搜索

名称	修改日期	类型	大小
 Qt5Mqtt.dll	2024/1/9 21:05	应用程序扩展	186 KB
Qt5Mqttd.dll	2024/1/9 21:05	应用程序扩展	510 KB

2. 部署Mqtt库到Qt安装目录

将源码编译生成目录lib下的6个文件复制到Qt安装目录内。

» 02-ClassNotes » emb2307 » 13-linuxdriver » 28-qtmqtt » build-qtmqtt-Desktop_Qt_5_15_2_MSVC2019_64bit-Release » lib

在 lib 中搜索

名称	修改日期	类型	大小
cmake	2024/1/9 21:04	文件夹	
Qt5Mqtt.dll	2024/1/9 21:05	应用程序扩展	186 KB
Qt5Mqtt.exp	2024/1/9 21:05	Exports Library F...	101 KB
Qt5Mqtt.lib	2024/1/9 21:05	Object File Library	166 KB
Qt5Mqtt.pdb	2024/1/9 21:05	Program Debug ...	3,892 KB
Qt5Mqtt.prl	2024/1/9 21:04	PRL 文件	2 KB
Qt5Mqttd.dll	2024/1/9 21:05	应用程序扩展	510 KB
Qt5Mqttd.exp	2024/1/9 21:05	Exports Library F...	101 KB
Qt5Mqttd.lib	2024/1/9 21:05	Incremental Link...	4,563 KB
Qt5Mqttd.lib	2024/1/9 21:05	Object File Library	167 KB
Qt5Mqttd.pdb	2024/1/9 21:05	Program Debug ...	4,540 KB
Qt5Mqttd.prl	2024/1/9 21:04	PRL 文件	2 KB

复制并粘贴到Qt安装目录下的lib文件夹中：

Software > Qt5.15.2 > 5.15.2 > msvc2019_64 > lib > 在 lib 中搜索

名称	修改日期	类型	大小
Qt5Location.prl	2020/11/6 17:02	PRL 文件	2 KB
Qt5Locationd.lib	2020/11/10 18:48	Object File Library	1,712 KB
Qt5Locationd.prl	2023/12/1 15:29	PRL 文件	2 KB
Qt5Mqtt.dll	2024/1/9 21:05	应用程序扩展	186 KB
Qt5Mqtt.lib	2024/1/9 21:05	Object File Library	166 KB
Qt5Mqtt.prl	2024/1/9 21:04	PRL 文件	2 KB
Qt5Mqttd.dll	2024/1/9 21:05	应用程序扩展	510 KB
Qt5Mqttd.lib	2024/1/9 21:05	Object File Library	167 KB
Qt5Mqttd.prl	2024/1/9 21:04	PRL 文件	2 KB
Qt5Multimedia.lib	2020/11/6 17:02	Object File Library	822 KB
Qt5Multimedia.prl	2023/12/1 15:29	PRL 文件	2 KB
Qt5Multimediad.lib	2020/11/6 17:02	Object File Library	824 KB
Qt5Multimediad.prl	2023/12/1 15:29	PRL 文件	2 KB
Qt5MultimediaQuick.lib	2020/11/6 17:03	Object File Library	37 KB

接下来将源码编译生成的两个库文件拷贝到Qt安装目录的bin中：

02-ClassNotes
 emb2307
 13-linuxdriver
 28-qtmqtt
 build-qtmqtt-Desktop_Qt_5_15_2_MSVC2019_64bit-Release
 bin

名称	修改日期	类型	大小
Qt5Mqtt.dll	2024/1/9 21:05	应用程序扩展	186 KB
Qt5Mqttd.dll	2024/1/9 21:05	应用程序扩展	510 KB

复制并粘贴Qt安装目录下的bin文件夹中。

SoftwareQt5.15.25.15.2msvc2019_64includeQtMqtt

名称	修改日期	类型	大小
qmqttauthenticationproperties.h	2020/10/27 21:07	C++ Header file	3 KB
QMqttClient	2024/1/9 22:07	文件	1 KB
qmqttdclient.h	2020/10/27 21:07	C++ Header file	9 KB
qmqttdclient_p.h	2020/10/27 21:07	C++ Header file	3 KB

QMqttClient - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

#include "qmqttdclient.h"

第 1 行, 第 25 列100%Windows (CRLF)UTF-8

qmqttdtopicfilter.h	2020/10/27 21:07	C++ Header file	4 KB
qmqttdtopicname.h	2020/10/27 21:07	C++ Header file	4 KB
qmqttdtype.h	2020/10/27 21:07	C++ Header file	3 KB

再次编译（Release模式）工程，编译通过。

mainwindow.h @ simpleclient - Qt Creator

文件(F) 编辑(E) 视图(V) 构建(B) 调试(D) 分析(A) 工具(T) 控件(W) 帮助(H)

项目

simpleclient

simpleclient.pro

头文件

mainwindow.h

源文件

main.cpp

mainwindow.cpp

界面文件

mainwindow.ui

1

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

/******

#ifndef MAINWINDOW_H

#define MAINWINDOW_H

#include <QMainWindow>

// #include <QMqttClient>

#include <QMqtt/QMqttClient>

QT_BEGIN_NAMESPACE

namespace Ui {

class MainWindow;

};

QT_END_NAMESPACE

class MainWindow : public QMainWindow

编译输出

w34100 -w34189 -w44996 -w44456 -w44457 -w44458 -wd4577 -wd4467 -EHsc -DUNICODE -D_UNICODE -DWIN32 -D_ENABLE_EXTENDED_ALIGNED_STORAGE -DWIN64 -DQT_DEPRECATED_WARNINGS -DDEBUG -DQT_NO_DEBUG -DQT_MQTT_LIB -DQT_WIDGETS_LIB -DQT_GUI_LIB -DQT_NETWORK_LIB -DQT_CORE_LIB -I..\\..\\..\\simpleclient -I..\\IE:/YShengli/02-ClassNotes/emb2307/13-linuxdriver/28-qtmqtt/qt5mqtt-5.15.2/include -IE:/YShengli/02-ClassNotes/emb2307/13-linuxdriver/28-qtmqtt/qt5mqtt-5.15.2/include/Qtmqtt -I..\\..\\..\\build-qtmqtt-Desktop_Qt_5_15_2_MSVC2019_64bit-Release/include -I..\\..\\..\\build-qtmqtt-Desktop_Qt_5_15_2_MSVC2019_64bit-Release/include/Qtmqtt -ID: \\Software\\Qt5.15.2\\5.15.2\\msvc2019_64\\include -ID: \\Software\\Qt5.15.2\\5.15.2\\msvc2019_64\\include\\QtWidgets -ID: \\Software\\Qt5.15.2\\5.15.2\\msvc2019_64\\include\\QtGui -ID: \\Software\\Qt5.15.2\\5.15.2\\msvc2019_64\\include\\QtANGLE -ID: \\Software\\Qt5.15.2\\5.15.2\\msvc2019_64\\include\\QtNetwork -ID: \\Software\\Qt5.15.2\\5.15.2\\msvc2019_64\\include\\QtCore -Irelease -I. -I/include -ID: \\Software\\Qt5.15.2\\5.15.2\\msvc2019_64\\mkspecs\\win32-msvc -Forelease\\ @C: \\Users\\sheng\\AppData\\Local\\Temp\\moc_mainwindow.obj.18944.422.jom

link /NOLOGO /DYNAMICBASE /NXCOMPAT /OPT:REF /INCREMENTAL:NO /SUBSYSTEM:WINDOWS "/

MANIFESTDEPENDENCY:type='win32' name='Microsoft.Windows.Common-Controls' version='6.0.0.0'

publicKeyToken='6595b64144ccf1df' language='*' processorArchitecture='*' /MANIFEST:embed /

OUT:release\\simplemqttclient.exe @C: \\Users\\sheng\\AppData\\Local\\Temp\\simplemqttclient.exe.18944.1266.jom

22:09:34: 进程"D: \\Software\\Qt5.15.2\\Tools\\QtCreator\\bin\\jom\\jom.exe"正常退出。

22:09:34: Elapsed time: 00:02.

构建

执行可执行程序。

MainWindow

File

Host:

Port: 1883

Topic: qtmqtt/topic1

Message: This is a test message

Log Messages

周二 1月 9 22:11:22 2024: State Change0

Quit